

■ Terminologia

análise de dados de pesquisa 180
 array de subscritos múltiplos 188
 array *m*-por-*n* 188
 arrays 161
 arrays de subscritos duplos 188
 arrays multidimensionais 188
 bubble sort 179
 caractere nulo 171
 chave de pesquisa 184
 const 176
 constante simbólica 166
 elementos 161
 elemento zerésimo 161
 escalares 176

escaláveis 166
 índice (ou subscrito) 162
 inicializadores 164
 nome 162
 número de posição 161
 pesquisa 184
 pesquisa binária 184
 pesquisa linear 184
 sinking sort 179
 subscrito 162
 tabelas 188
 texto substituto 166
 valor 162
 valor de chave 184

■ Exercícios de autorrevisão

6.1 Preencha os espaços em cada uma das sentenças a seguir:

- a)** Listas e tabelas de valores são armazenadas em _____.
- b)** Os elementos de um array têm em comum o fato de que possuem os(as) mesmos(as) _____ e _____.
- c)** O número de referência a um elemento em particular de um array é chamado seu _____.
- d)** Um(a) _____ deverá ser usado(a) para especificar o tamanho de um array, pois torna o programa mais escalável.
- e)** O processo de colocar os elementos de um array em ordem é chamado _____ do array.
- f)** Determinar se um array contém certo valor de chave é chamado _____ o array.
- g)** Um array que usa dois subscritos é conhecido como array de _____.

6.2 Indique se as sentenças a seguir são *falsas* ou *verdadeiras*. Caso a resposta seja *falsa*, explique o motivo.

- a)** Um array pode armazenar muitos tipos diferentes de valores.
- b)** Um subscrito de array pode ser do tipo de dado `double`.
- c)** Se houver menos valores em uma lista de inicializadores que o número de elementos no array, C automaticamente inicializará os elementos restantes até o último valor na lista de inicializadores.
- d)** Consiste em erro o fato de uma lista de inicializadores ter mais valores que o número de elementos no array.

- e)** Um elemento individual do array que é passado a uma função como argumento na forma `a[i]` e modificado na função chamada terá o valor modificado na função que a chamou.

6.3 Considere um array chamado `fractions`.

- a)** Defina uma constante simbólica `SIZE` que possa ser substituída pelo texto substituto 10.
- b)** Defina um array com elementos `SIZE` do tipo `double` e inicialize-os em 0.
- c)** Referencie o quarto elemento a partir do início do array.
- d)** Referencie o elemento 4 do array.
- e)** Atribua o valor 1,667 ao elemento 9 do array.
- f)** Atribua o valor 3,333 ao sétimo elemento do array.
- g)** Imprima os elementos 6 e 9 do array com dois dígitos de precisão à direita do ponto decimal e mostre a saída que aparece na tela.
- h)** Imprima todos os elementos do array, usando uma estrutura de repetição `for`. Suponha que a variável inteira `x` tenha sido definida como uma variável de controle para o loop. Mostre a saída.

6.4 Escreva comandos que realizem as seguintes tarefas:

- a)** Definam `table` como um array de inteiros com 3 linhas e 3 colunas. Suponha que a constante simbólica `SIZE` tenha sido definida como 3.
- b)** Quantos elementos o array `table` contém? Imprima o número total de elementos.
- c)** Use uma estrutura de repetição `for` para inicializar cada elemento de `table` como a soma de seus

subscritos. Suponha que as variáveis inteiras *x* e *y* sejam definidas como variáveis de controle.

- d) Imprima os valores de cada elemento do array *table*. Suponha que o array tenha sido inicializado com a definição:

```
int table[ SIZE ][ SIZE ] =
    { { 1, 8 }, { 2, 4, 6 }, { 5 }
};
```

- 6.5 Identifique e corrija os erros em cada um dos segmentos de programa a seguir.

a) `#define SIZE 100;`

b) `SIZE = 10;`

c) Considere `int b[ 10 ] = { 0 }, i;`

```
for ( i = 0; i <= 10; i++ ) {
    b[ i ] = 1;
}
```

d) `#include <stdio.h>;`

e) Considere `int a[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };`

`a[ 1, 1 ] = 5;`

f) `#define VALUE = 120`

Respostas dos exercícios de autorrevisão

- 6.1 a) Arrays. b) Nome, tipo. c) Subscrito. d) Constante simbólica. e) Ordenação. f) Pesquisar. g) Subscrito duplo.

- 6.2 a) Falso. Um array pode armazenar somente valores do mesmo tipo.

b) Falso. Um subscrito de array precisa ser um inteiro ou uma expressão inteira.

c) Falso. C inicializa automaticamente os elementos restantes em zero.

d) Verdadeiro.

e) Falso. Os elementos individuais de um array são passados por valor. Se um array inteiro for passado a uma função, então quaisquer modificações serão refletidas no original.

- 6.3 a) `#define SIZE 10`

b) `double fractions[ SIZE ] = { 0.0 };`

c) `fractions[ 3 ]`

d) `fractions[ 4 ]`

e) `fractions[ 9 ] = 1.667;`

f) `fractions[ 6 ] = 3.333;`

g) `printf( "%.2f %.2f\n", fractions[ 6 ], fractions[ 9 ] );`

Saída: 3.33 1.67.

h) `for ( x = 0; x < SIZE; x++ ) {`

```
    printf( "fractions[%d] = %f\n", x,
fractions[ x ] );
}
```

Saída:

```
fractions[0] = 0.000000
fractions[1] = 0.000000
fractions[2] = 0.000000
```

```
fractions[3] = 0.000000
```

```
fractions[4] = 0.000000
```

```
fractions[5] = 0.000000
```

```
fractions[6] = 3.333000
```

```
fractions[7] = 0.000000
```

```
fractions[8] = 0.000000
```

```
fractions[9] = 1.667000
```

- 6.4 a) `int table[ SIZE ][ SIZE ];`

b) Nove elementos. `printf( "%d\n", SIZE * SIZE );`

c) `for ( x = 0; x < SIZE; x++ ) {`

```
    for ( y = 0; y < SIZE; y++ ) {
```

```
        table[ x ][ y ] = x + y;
```

```
    }
```

```
}
```

d) `for ( x = 0; x < SIZE; x++ ) {`

```
    for ( y = 0; y < SIZE; y++ ) {
```

```
        printf( "table[%d][%d] = %d\n", x, y,
table[ x ][ y ] );
```

```
    }
```

```
}
```

Saída:

```
table[0][0] = 1
```

```
table[0][1] = 8
```

```
table[0][2] = 0
```

```
table[1][0] = 2
```

```
table[1][1] = 4
```

```
table[1][2] = 6
```

```
table[2][0] = 5
```

```
table[2][1] = 0
```

```
table[2][2] = 0
```

- 6.5** a) Erro: Ponto e vírgula no final da diretiva do pré-processador `#define`.

Correção: Elimine o ponto e vírgula.

- b) Erro: Atribuição de um valor a uma constante simbólica usando uma instrução de atribuição.

Correção: Atribua um valor à constante simbólica em uma diretiva do pré-processador `#define` sem usar o operador de atribuição, como em `#define SIZE 10`.

- c) Erro: Referência a um elemento do array fora dos limites do array (`b[ 10 ]`).

Correção: Altere o valor final da variável de controle para 9.

- d) Erro: Ponto e vírgula no final da diretiva do pré-processador `#include`.

Correção: Elimine o ponto e vírgula.

- e) Erro: Subscrito do array feito incorretamente.

Correção: Mude a instrução para `a[ 1 ][ 1 ] = 5;`

- f) Erro: Atribuição de um valor a uma constante simbólica usando uma instrução de atribuição.

Correção: Atribua um valor à constante simbólica em uma diretiva do pré-processador `#define` sem usar o operador de atribuição, como em `#define VALUE 120`.

Exercícios

- 6.6** Preencha os espaços em cada uma das sentenças a seguir:

- C armazena listas de valores em _____.
- Os elementos de um array têm em comum o fato de que eles _____.
- Ao se referir a um elemento do array, o número da posição contido dentro dos parênteses é um(a) _____.
- Os nomes dos cinco elementos do array `p` são _____, _____, _____, _____ e _____.
- O conteúdo de um elemento em particular de um array é chamado _____ desse elemento.
- Dar nome a um array, indicar seu tipo e especificar o número de elementos contidos nele é chamado _____ o array.
- O processo de colocar os elementos de um array em ordem crescente ou decrescente é chamado _____.
- Em um array de subscrito duplo, o primeiro subscrito (por convenção) identifica a _____ de um elemento, e o segundo subscrito (por convenção) identifica a _____ de um elemento.
- Um array *m*-por-*n* contém _____ linhas, _____ colunas e _____ elementos.
- O nome do elemento do array `d` na linha 3, coluna 5 é _____.

- 6.7** Indique quais das afirmações a seguir são *verdadeiras* e quais são *falsas*. No caso de afirmações *falsas*, explique.

- Para se referir a determinado local ou elemento dentro de um array, especificamos o nome do array e o valor do elemento em particular.
- Uma definição de array reserva espaço para o array.
- Para indicar que 100 locais devem ser reservados para o array de inteiros `p`, escreva

```
p[ 100 ];
```

- Um programa em C que inicializa os elementos de um array de 15 elementos em zero precisa conter somente uma estrutura `for`.
- Um programa em C que totaliza os elementos de um array com subscrito duplo precisa conter estruturas `for` aninhadas.
- A média, a mediana e a moda do conjunto de valores 1, 2, 5, 6, 7, 7, 7 são, respectivamente, 5, 6 e 7.

- 6.8** Escreva instruções que realizem as seguintes tarefas:

- Exibam o valor do sétimo elemento do array de caracteres `f`.
- Incluam um valor no elemento 4 do array de ponto flutuante com o subscrito único `b`.
- Inicializem cada um dos cinco elementos do array de inteiros com o subscrito único `g` em 8.
- Somem os elementos do array de ponto flutuante `c` de 100 elementos.
- Copiem o array `a` na primeira parte do array `b`. Considere `double a[11], b[34];`
- Determinem e imprimam o menor e o maior valor contidos no array de ponto flutuante com 99 elementos `w`.

- 6.9** Considere um array de inteiros `t` de 2 por 5.

- Escreva uma definição para `t`.
- Quantas linhas `t` possui?
- Quantas colunas `t` possui?
- Quanto elementos `t` possui?
- Escreva os nomes de todos os elementos na segunda linha de `t`.
- Escreva os nomes de todos os elementos na terceira coluna de `t`.
- Escreva uma única instrução que defina o elemento de `t` na linha 1, coluna 2, como zero.
- Escreva uma série de instruções que inicializem cada elemento de `t` em zero. Não use uma estrutura de repetição.

- i) Escreva uma estrutura *for* aninhada que inicialize cada elemento de *t* em zero.
- j) Escreva uma instrução que insira os valores dos elementos de *t* a partir do terminal.
- k) Escreva uma série de instruções que determinem e imprimam o menor valor no array *t*.
- l) Escreva uma instrução que apresente os elementos da primeira linha de *t*.
- m) Escreva uma instrução que some os elementos da quarta coluna de *t*.
- n) Escreva uma série de instruções que imprimam o array *t* em formato tabular. Liste os subscritos de coluna como cabeçalhos no topo e liste os subscritos de linha à esquerda de cada linha.

6.10 Comissões de vendas. Use um array com subscrito único para resolver o problema a seguir. Uma empresa paga o salário de seus vendedores com base em uma comissão. O vendedor recebe R\$ 200,00 por semana, e mais 9 por cento de suas vendas brutas nessa semana. Por exemplo, um vendedor, que totalize R\$ 3.000,00 em vendas em uma semana, receberá R\$ 200,00 e mais 9 por cento de R\$ 3.000,00, ou seja, R\$ 470,00. Escreva um programa em C (usando um array de contadores) que determine quantos vendedores receberam salários dentro de cada um dos seguintes intervalos (suponha que o salário de cada vendedor seja arredondado em um valor inteiro):

- a) R\$ 200–299
- b) R\$ 300–399
- c) R\$ 400–499
- d) R\$ 500–599
- e) R\$ 600–699
- f) R\$ 700–799
- g) R\$ 800–899
- h) R\$ 900–999
- i) R\$ 1000 ou mais

6.11 Bubble sort. O bubble sort apresentado na Figura 6.15 é ineficaz no caso de arrays grandes. Faça as modificações simples descritas a seguir para melhorar o desempenho do bubble sort.

- a) Após a primeira passada, o maior número estará certamente no elemento de número mais alto do array; após a segunda passada, os dois maiores números estarão ‘no lugar’ e assim por diante. Em vez de nove comparações em cada passada, modifique o bubble sort para que sejam feitas oito comparações na segunda passada, sete na terceira e assim por diante.
- b) Os dados no array talvez já estejam na ordem correta, ou na ordem quase correta, portanto, por que fazer nove passadas se menos que isso já seria suficiente? Modifique a ordenação para verificar, ao

final de cada passada, se alguma troca foi feita. Se nenhuma troca tiver sido feita, então os dados já deverão estar na ordem certa, de modo que o programa deverá ser concluído. Se foram feitas trocas, então, pelo menos mais uma passada será necessária.

6.12 Escreva instruções isoladas que realizem cada uma das operações de array de subscrito único a seguir:

- a) Inicializem os 10 elementos do array de inteiros *counts* com zeros.
- b) Somem 1 a cada um dos 15 elementos do array de inteiros *bonus*.
- c) Leiam os 12 valores do array de ponto flutuante *tempPorMes* via teclado.
- d) Imprimam os cinco valores do array de inteiros *bestScores* em formato de coluna.

6.13 Ache o(s) erro(s) em cada uma das seguintes instruções:

- a) Considere: `char str[ 5 ];`
`scanf( "%s", str ); /* Usuário digita olá */`
- b) Considere: `int a[ 3 ];`
`printf( "$d%d%d\n", a[ 1 ], a[ 2 ], a[ 3 ] );`
- c) `double f[ 3 ] = { 1.1, 10.01, 100.001, 1000.0001 };`
- d) Considere: `double d[ 2 ][ 10 ];`
`d[ 1, 9 ] = 2.345;`

6.14 Modificações nos programas de média, mediana e moda. Modifique o programa da Figura 6.16 de modo que a função *mode* seja capaz de lidar com um empate para o valor de moda. Modifique também a função *median* para que os dois elementos do meio de um array tenham a média calculada caso haja um número par de elementos.

6.15 Eliminação de duplicatas. Use um array de subscrito único para resolver o problema a seguir. Leia 20 números, cada um entre 10 e 100, inclusive. À medida que cada número for lido, imprima-o apenas se ele não for uma duplicata de um número já lido. Considere a ‘pior das hipóteses’: os 20 números são diferentes. Use o menor array possível para resolver esse problema.

6.16 Rotule os elementos do array *3* por *5* de subscrito duplo *sales* para indicar a ordem em que eles são definidos em zero pelo segmento de programa a seguir:

```
for ( row = 0; row <= 2; row++ ) {
    for ( column = 0; column <= 4;
        column++ ) {
        sales[ row ][ column ] = 0;
    }
}
```

6.17 O que o programa a seguir faz?

```

1 /* ex06_17.c */
2 /* O que esse programa faz? */
3 #include <stdio.h>
4 #define SIZE 10
5
6 int whatIsThis( const int b[], int p ); /*
   protótipo de função */
7
8 /* função main inicia a execução do progra-
   ma */
9 int main( void )
10 {
11     int x; /* mantém valor de retorno da fun-
   ção whatIsThis */
12
13     /* inicializa array a */
14     int a[ SIZE ] = { 1, 2, 3, 4, 5, 6, 7,
15                       8, 9, 10 };
16
17     x = whatIsThis( a, SIZE );
18
19     printf( "Resultado é %d\n", x );
20     return 0; /* indica conclusão bem-sucedida */
21 } /* fim do main */
22
23 /* O que essa função faz? */
24 int whatIsThis( const int b[], int p )
25 {
26     /* caso básico */
27     if ( p == 1 ) {
28         return b[ 0 ];
29     } /* fim do if */
30
31     /* etapa de recursão */
32     return b[ p - 1 ] + whatIsThis( b, p - 1 );
33 } /* fim do else */
34 } /* fim da função whatIsThis */

```

6.18 O que o programa a seguir faz?

```

1 /* ex06_18.c */
2 /* O que esse programa faz? */
3 #include <stdio.h>
4 #define SIZE 10
5
6 /* protótipo de função */
7 void someFunction( const int b[], int start-
   tIndex, int size );
8
9 /* função main inicia a execução do progra-
   ma */
10 int main( void )
11 {

```

```

12     int a[ SIZE ] = { 8, 3, 1, 2, 6, 0, 9, 7,
13                       4, 5 }; /* inicializa a */
14
15     printf( "Resposta é:\n" );
16     someFunction( a, 0, SIZE );
17     printf( "\n" );
18     return 0; /* indica conclusão bem-suce-
   dida */
19 } /* fim do main */
20
21 /* O que essa função faz? */
22 void someFunction( const int b[], int start-
   tIndex, int size )
23 {
24     if ( startIndex < size ) {
25         someFunction( b, startIndex + 1, size
26                       );
27         printf( "%d ", b[ startIndex ] );
28     } /* fim do if */
29 } /* fim da função someFunction */

```

6.19 Lançando dados. Escreva um programa que simule o lançamento de dois dados. O programa deverá usar `rand` para lançar o primeiro dado, e deverá usar `rand` novamente para lançar o segundo dado. Em seguida, a soma dos dois valores deverá ser calculada. [Nota: como cada dado pode mostrar um valor inteiro de 1 a 6, então, a soma dos dois valores variará de 2 a 12, com 7 sendo o resultado mais frequente; e 2 e 12, os resultados menos frequentes.] A Figura 6.23 mostra as 36 combinações possíveis dos dois dados. Seu programa deverá lançar os dois dados 36.000 vezes. Use um array de subscrito único para contar o número de vezes em que cada resultado possível aparece. Imprima os resultados em um formato tabular. Além disso, determine se os resultados são razoáveis; ou seja, existem seis maneiras de somar um 7, de modo que um sexto de todas as jogadas, aproximadamente, deverá ser 7.

	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

Figura 6.23 ■ Resultados do lançamento de dois dados.

6.20 *Jogo de craps.* Escreva um programa que execute 1000 partidas de craps (sem intervenção humana) e responda a cada uma das perguntas a seguir:

- a) Quantos jogos são ganhos no primeiro, no segundo, ..., no vigésimo lançamento e após o vigésimo lançamento?
- b) Quantos jogos são perdidos no primeiro, no segundo, ..., no vigésimo lançamento e após o vigésimo lançamento?
- c) Quais são as chances de se ganhar no craps? [Nota: você descobrirá que craps é um dos jogos mais honestos do cassino. O que você acha que isso significa?]
- d) Qual a média de lançamentos em um jogo de craps?
- e) As chances de vencer crescem se o jogo se prolongar?

6.21 *Sistema de reservas de passagens aéreas.* Uma pequena companhia aérea acabou de comprar um computador para seu novo sistema automatizado de reservas. O presidente lhe pediu que programasse o novo sistema. Você escreverá um programa que atribuirá assentos em cada voo do único avião da companhia (capacidade: 10 assentos).

Seu programa deverá exibir o seguinte menu de alternativas:

Favor digitar 1 para “primeira classe”

Favor digitar 2 para “econômica”

Se a pessoa digitar 1, então seu programa deverá designar um assento na primeira classe (assentos de 1 a 5). Se a pessoa digitar 2, então seu programa deverá designar um assento na classe econômica (assentos 6 a 10). Seu programa deverá, então, imprimir um bilhete de embarque que indique o número do assento da pessoa e a seção a que esse assento pertence, primeira classe ou classe econômica.

Use um array de subscrito único para representar o quadro de assentos do avião. Inicialize todos os elementos do array em 0 para indicar que todos os assentos estão vazios. À medida que cada assento é designado, defina o elemento correspondente do array como 1, para indicar que o assento não está mais disponível.

Seu programa, naturalmente, nunca deverá designar um assento que já foi reservado a outra pessoa. Quando a primeira classe estiver cheia, seu programa deverá perguntar à pessoa se ela aceita ser colocada na classe econômica (e vice-versa). Se a resposta for sim, faça a designação de assentos apropriada. Se a resposta for não, imprima a mensagem “O próximo voo sairá em 3 horas.”

6.22 *Total de vendas.* Use um array de subscrito duplo para resolver o problema a seguir. Uma empresa tem quatro vendedores (1 a 4) que vendem cinco produtos diferentes (1 a 5). Uma vez por dia, cada vendedor passa uma nota para cada tipo diferente de produto vendido. Cada nota contém:

- a) O número do vendedor.
- b) O número do produto.
- c) O valor em reais desse produto vendido nesse dia.

Assim, cada vendedor passa entre 0 e 5 notas de vendas por dia. Considere que as informações de todas as notas passadas no mês anterior estejam disponíveis. Escreva um programa que leia todas as informações das vendas do mês passado e resuma o total de vendas por vendedor por produto. Todos os totais devem ser armazenados em um array com subscrito duplo `sales`. Depois de processar todas as informações do mês passado, imprima os resultados em formato tabular, com cada uma das colunas representando um vendedor em particular, e cada uma das linhas representando um produto em particular. Calcule o total de cada linha para obter o total de vendas de cada produto no mês passado; cruze os resultados obtidos com os de cada coluna para obter o total de vendas por vendedor no mês passado. Seu relatório tabular deverá incluir os totais à direita das linhas e no final das colunas.

6.23 *Gráficos de tartaruga.* A linguagem Logo, que é particularmente popular entre os usuários de computadores pessoais, tornou famoso o conceito de *gráficos de tartaruga*. Imagine uma tartaruga mecânica que caminhe pela sala sob o controle de um programa em C. A tartaruga segura uma caneta em uma de duas posições, para cima ou para baixo. Quando a caneta está para baixo, a tartaruga traça as formas enquanto se movimenta; quando a caneta está para cima, a tartaruga se movimenta livremente, sem escrever nada. Neste problema, você simulará a operação da tartaruga e também criará um bloco de rascunho computadorizado.

Use um array de 50 por 50 `floor` que seja inicializado em zeros. Leia comandos de um array em que eles estejam contidos. Registre a posição atual da tartaruga o tempo todo, e se a caneta está para cima ou para baixo. Suponha que a tartaruga sempre comece na posição 0,0 do piso com sua caneta para cima. O conjunto de comandos da tartaruga que seu programa deverá processar aparece na Figura 6.24. Suponha que a tartaruga esteja em algum ponto próximo do centro do piso. O ‘programa’ a seguir desenharia e imprimiria um quadrado de 12 por 12:

2
5.12
3
5.12
3
5.12
3
5.12
1
6
9

Enquanto a tartaruga se mover com a caneta para baixo, defina os elementos apropriados do array `floor` como 1. Quando o comando 6 (imprimir) for dado, onde houver um 1 no array, mostre um asterisco ou algum outro caractere à sua escolha. Sempre que houver um zero, mostre um espaço em branco. Escreva um programa que implemente as capacidades dos gráficos de tartaruga discutidas aqui. Escreva vários programas de gráficos de tartaruga para desenhar formas interessantes. Acrescente outros comandos para aumentar o poder de sua linguagem de gráficos de tartaruga.

Comando	Significado
1	Caneta para cima
2	Caneta para baixo
3	Virar à direita
4	Virar à esquerda
5, 10	Move 10 espaços adiante (ou algum número diferente de 10)
6	Imprime o array de 50 por 50 array
9	Fim dos dados (sentinela)

Figura 6.24 ■ Comandos da tartaruga.

6.24 Passeio do cavalo. Um dos desafios mais interessantes que os jogadores de xadrez enfrentam é o problema do Passeio do Cavalo, proposto originalmente pelo matemático Euler. O problema é o seguinte: a peça do xadrez chamada cavalo pode se movimentar por um tabuleiro vazio e tocar cada um dos 64 quadrados uma e somente uma vez? Estudaremos esse problema interessante em detalhes.

O cavalo se movimenta em forma de L (por duas casas em uma direção e depois por uma casa em uma direção perpendicular). Assim, por um quadrado no meio de um tabuleiro vazio, o cavalo pode fazer oito movimentos diferentes (numerados de 0 a 7), como mostra a Figura 6.25.

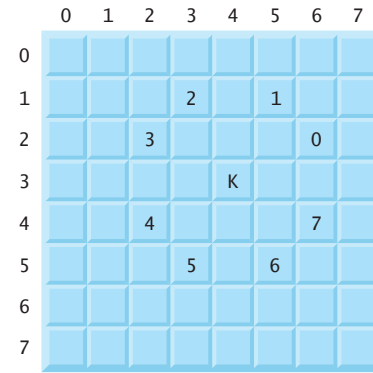


Figura 6.25 ■ Os oito movimentos possíveis do cavalo.

- a) Desenhe um tabuleiro de 8 por 8 em uma folha de papel e experimente o Passeio do Cavalo manualmente. Coloque 1 no primeiro quadrado para onde você moveu o cavalo, 2 no segundo quadrado, 3 no terceiro e assim por diante. Antes de iniciar o passeio, estime a que distância você acha que chegará, lembrando-se de que um passeio completo consiste em 64 movimentos. A que distância você chegará? Você chegou perto do resultado estimado?
- b) Agora, vamos desenvolver um programa que movimentará o cavalo em um tabuleiro. O tabuleiro em si é representado por um array com subscrito duplo de 8 por 8 `board`. Cada um dos quadrados é inicializado em zero. Descreveremos cada um dos oito movimentos possíveis em termos de seus componentes horizontal e vertical. Por exemplo, um movimento do tipo 0, como mostra a Figura 6.25, consiste em mover dois quadrados horizontalmente para a direita e um quadrado verticalmente para cima. O movimento 2 consiste em mover um quadrado horizontalmente para a esquerda e dois quadrados verticalmente para cima. Movimentos horizontais à esquerda e verticais para cima são indicados por números negativos. Os oito movimentos podem ser descritos por dois arrays de subscrito único, `horizontal` e `vertical`, da seguinte forma:

```
horizontal[ 0 ] = 2
horizontal[ 1 ] = 1
horizontal[ 2 ] = -1
horizontal[ 3 ] = -2
horizontal[ 4 ] = -2
horizontal[ 5 ] = -1
horizontal[ 6 ] = 1
horizontal[ 7 ] = 2
```

```

vertical[ 0 ] = -1
vertical[ 1 ] = -2
vertical[ 2 ] = -2
vertical[ 3 ] = -1
vertical[ 4 ] = 1
vertical[ 5 ] = 2
vertical[ 6 ] = 2
vertical[ 7 ] = 1

```

Considere que as variáveis `currentRow` e `currentColumn` indiquem a linha e coluna da posição atual do cavalo no tabuleiro. Para fazer um movimento do tipo `moveNumber`, onde `moveNumber` está entre 0 e 7, seu programa usa as instruções

```

currentRow += vertical[ moveNumber ];
currentColumn += horizontal[ moveNumber ];

```

Mantenha um contador que varie de 1 a 64. Registre o maior contador em cada quadrado para onde o cavalo se mover. Lembre-se de testar cada movimento em potencial para ver se o cavalo já visitou esse quadrado. E, naturalmente, teste cada movimento potencial para garantir que o cavalo não saia do tabuleiro. Agora, escreva um programa para mover o cavalo pelo tabuleiro. Execute o programa. Quantos movimentos o cavalo fez?

- c) Depois de tentar escrever e executar um programa do Passeio do Cavalo, você provavelmente terá desenvolvido algumas ideias valiosas. Vamos usá-las para desenvolver uma *heurística* (ou estratégia) para movimentar o cavalo. A heurística não garante o sucesso, mas uma heurística cuidadosamente desenvolvida melhora bastante as chances de sucesso. Talvez você tenha observado que os quadrados externos, de certa forma, são mais trabalhosos do que os mais próximos do centro do tabuleiro. Na verdade, os quadrados mais problemáticos, ou inacessíveis, são os dos quatro cantos.

A intuição pode sugerir que você deva tentar mover o cavalo para os quadrados mais problemáticos primeiro e deixar em aberto aqueles que são mais acessíveis, de modo que, quando o tabuleiro ficar congestionado, mais para o final do passeio, haverá uma chance maior de sucesso.

Podemos desenvolver uma 'heurística de acessibilidade', classificando cada um dos quadrados de acordo com sua facilidade de acesso, e sempre movendo o cavalo para o quadrado (dentro dos movimentos permitidos, é claro) mais inacessível. Rotulamos um array de duplo subscrito `accessibility` com números indicando a partir de quantos quadrados cada

quadrado em particular é acessível. Em um tabuleiro vazio, os quadrados do centro, portanto, são classificados como 8, os quadrados do canto são classificados como 2, e os outros quadrados têm números de acessibilidade 3, 4 ou 6, como mostrado a seguir:

2	3	4	4	4	4	3	2
3	4	6	6	6	6	4	3
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
3	4	6	6	6	6	4	3
2	3	4	4	4	4	3	2

Agora escreva uma versão do programa Passeio do Cavalo usando a heurística de acessibilidade. A qualquer momento, o cavalo deverá se mover para o quadrado com o menor número de acessibilidade. No caso de um empate, o cavalo pode se mover para qualquer um dos quadrados com a mesma classificação. Portanto, o passeio pode começar em qualquer um dos quatro cantos. [Nota: à medida que o cavalo se move pelo tabuleiro, seu programa deverá reduzir os números de acessibilidade quando mais e mais quadrados forem ocupados. Desse modo, em qualquer momento durante o passeio, o número de acessibilidade de cada quadrado disponível continuará sendo igual a exatamente o número de quadrados dos quais esse quadrado pode ser alcançado.] Execute essa versão de seu programa. Você consegue fazer um passeio completo? Agora, modifique o programa para realizar 64 passeios, um a partir de cada quadrado do tabuleiro. Quantos passeios completos você fez?

- d) Escreva uma versão do programa Passeio do Cavalo que, ao encontrar um empate entre dois ou mais quadrados, decida qual quadrado escolher ao considerar o que poderia acontecer adiante, verificando os quadrados alcançáveis pelos quadrados 'empatados'. Seu programa deverá se mover para o quadrado pelo qual o próximo movimento alcançará um quadrado com o menor número de acessibilidade.

6.25 Passeio do cavalo: técnicas de força bruta. No Exercício 6.24, desenvolvemos uma solução para o problema do Passeio do Cavalo. A técnica usada, chamada 'heurística de acessibilidade', gera muitas soluções e é executado de modo eficiente.

À medida que os computadores continuarem crescendo em potência, poderemos resolver muitos problemas com o imenso poder da computação e de algoritmos relativamente pouco sofisticados. Vamos chamar essa técnica de solução do problema pela 'força bruta'.

- a) Use a geração de número aleatório para permitir que o cavalo se movimente pelo tabuleiro de xadrez aleatoriamente (logicamente, em seus movimentos legítimos em forma de L). Seu programa deverá executar um passeio e imprimir o tabuleiro final. Até onde o cavalo conseguiu chegar?
- b) Provavelmente, o programa anterior produziu um passeio relativamente curto. Agora, modifique seu programa para tentar 1000 passeios. Use um array de subscrito único para registrar o número de passeios de cada extensão. Quando seu programa terminar de experimentar os 1000 passeios, ele deverá mostrar essa informação em formato tabular. Qual foi o melhor resultado?
- c) Provavelmente, o programa anterior lhe ofereceu alguns passeios ‘respeitáveis’, mas não passeios completos. Agora, ‘retire todas as restrições’, e simplesmente deixe que seu programa seja executado até que produza um passeio completo. [Cuidado: essa versão do programa poderia ser executada por horas em um computador poderoso.] Mais uma vez, mantenha uma tabela dos números de passeios de cada extensão e imprima essa tabela quando o primeiro passeio completo for encontrado. Quantos passeios seu programa fez até produzir um passeio completo? Quanto tempo ele levou para fazer isso?
- d) Compare a versão força bruta do Passeio do Cavalo com a versão da heurística de acessibilidade. Qual das duas versões exigiu um estudo mais cuidadoso do problema? Qual algoritmo foi mais difícil de desenvolver? Qual exigiu mais poder do computador? Poderíamos estar certos (antecipadamente) de obter um passeio completo com a técnica de heurística de acessibilidade? Poderíamos estar certos (antecipadamente) de obter um passeio completo com a técnica de força bruta? Argumente os prós e os contras da solução geral do problema pela força bruta.

6.26 Oito rainhas. Outro desafio para os jogadores de xadrez é o problema das Oito Rainhas. Resumindo: é possível colocar oito rainhas em um tabuleiro vazio de modo que nenhuma rainha esteja ‘atacando’ qualquer outra – ou seja, de modo que duas rainhas nunca estejam na mesma linha, na mesma coluna ou na mesma diagonal? Use o tipo de raciocínio desenvolvido no Exercício 6.24 para formular uma heurística para a solução do problema das Oito Rainhas. Execute seu programa. [Dica: é possível atribuir um valor numérico a cada quadrado do tabuleiro, indicando quantos quadrados de um tabuleiro vazio são ‘eliminados’ quando uma rainha é colocada nesse quadrado. Por exemplo, cada um dos quatro cantos receberia o valor 22, como mostra a Figura 6.26.]

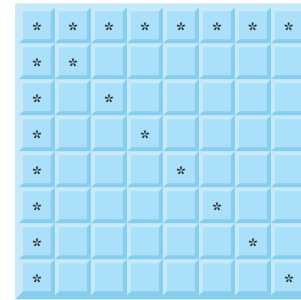


Figura 6.26 ■ Os 22 quadrados eliminados ao colocarmos uma rainha no canto superior esquerdo.

Quando esses ‘números de eliminação’ são colocados em todos os 64 quadrados, uma heurística apropriada poderia ser: coloque a próxima rainha no quadrado com o menor número de eliminação. Por que essa estratégia é intuitivamente atraente?

6.27 Oito rainhas: técnicas de força bruta. Neste problema, você desenvolverá várias técnicas de força bruta para solucionar o problema das Oito Rainhas, introduzindo no Exercício 6.26.

- a) Resolva o problema das Oito Rainhas usando a técnica de força bruta aleatória desenvolvida no Exercício 6.25.
- b) Use uma técnica exaustiva (ou seja, tente todas as combinações possíveis de oito rainhas no tabuleiro).
- c) Na sua opinião, por que a técnica de força bruta exaustiva pode não ser apropriada para resolver o problema das Oito Rainhas?
- d) Compare as técnicas de força bruta aleatória e de força bruta exaustiva em geral.

6.28 Eliminação de duplicatas. No Capítulo 12, exploraremos a estrutura de dados de árvore de pesquisa binária de alta velocidade. Um dos recursos de uma árvore de pesquisa binária é que os valores duplicados são descartados quando são feitas inserções na árvore. Isso é conhecido como eliminação de duplicatas. Escreva um programa que produza 20 números aleatórios entre 1 e 20. O programa deverá armazenar todos os valores não duplicados em um array. Use o menor array possível para conseguir realizar essa tarefa.

6.29 Passeio do cavalo: teste do passeio fechado. No Passeio do Cavalo, um passeio completo ocorre quando o cavalo faz 64 movimentos passando por quadrado do tabuleiro uma, e somente uma vez. Um passeio fechado ocorre quando o 64º movimento está a um movimento do local onde o cavalo iniciou o passeio. Modifique o programa do Passeio do Cavalo que você escreveu no Exercício 6.24 para testar um passeio fechado, se um passeio completo tiver ocorrido.

6.30 O crivo de Eratóstenes. Um número inteiro primo é qualquer inteiro maior que 1 que pode ser dividido apenas por ele mesmo e por 1. O crivo de Eratóstenes é um método para encontrar os números primos. Ele funciona da seguinte maneira:

- a) Crie um array com todos os elementos inicializados em 1 (verdadeiro). Os elementos do array com subscritos primos permanecerão em 1. Todos os outros elementos do array acabarão sendo definidos em zero.
- b) Começando com o subscrito de array 2 (o subscrito 1 não é primo), toda vez que um elemento do array cujo valor seja 1 for encontrado, percorra o restante do array e defina em zero cada elemento cujo sub-

crito seja um múltiplo do subscrito para o elemento com valor 1. Para o subscrito de array 2, todos os elementos no array, além de 2, que sejam múltiplos de 2 serão definidos em zero (subscritos, 4, 6, 8, 10, e assim por diante). Para o subscrito de array 3, todos os elementos no array, além de 3, que são múltiplos de 3 serão definidos em zero (subscritos 6, 9, 12, 15, e assim por diante).

Quando esse processo tiver sido concluído, os elementos do array que ainda estiverem definidos em 1 indicarão que o subscrito é um número primo. Escreva um programa que use um array de 1000 elementos para determinar e exibir os números primos entre 1 e 999. Ignore o elemento 0 do array.

Exercícios de recursão

6.31 *Palíndromos.* Um palíndromo é uma string que pode ser escrita da mesma forma nos dois sentidos, de frente para trás e de trás para a frente. Alguns exemplos de palíndromos são: 'radar', 'salta o atlas' e, se você ignorar espaços e acentos, 'Oto come doce seco de mocotó'. Escreva uma função recursiva `testPalindrome` que retorne 1 se a string armazenada no array for um palíndromo, e 0 se não for. A função deverá ignorar espaços e sinais de pontuação na string.

6.32 *Pesquisa linear.* Modifique o programa da Figura 6.18 para que seja possível usar uma função `linearSearch` recursiva na realização da pesquisa linear do array. A função deverá receber um array de inteiros e o tamanho do array como argumentos. Se a chave de pesquisa for encontrada, retorne o subscrito do array; caso contrário, retorne -1.

6.33 *Pesquisa binária.* Modifique o programa da Figura 6.19 para que seja possível usar uma função `binarySearch` recursiva na realização da pesquisa binária do array. A função deverá receber um array de inteiros e o subscrito inicial e final como argumentos. Se a chave de pesquisa for encontrada, retorne o subscrito do array; caso contrário, retorne -1.

6.34 *Oito rainhas.* Modifique o programa das Oito Rainhas que você criou no Exercício 6.26 para que possa resolver o problema recursivamente.

6.35 *Impressão de um array.* Escreva uma função recursiva `printArray` que apanhe um array e o tamanho do array como argumentos, imprima o array e não retorne nada. A função deverá encerrar o processamento e retornar quando receber um array de tamanho zero.

6.36 *Impressão de uma string ao contrário.* Escreva uma função recursiva `stringReverse` que apanhe um array de caracteres como argumento, imprima-o de trás para a frente e não retorne nada. A função deverá encerrar o processamento e retornar quando o caractere nulo de finalização da string for encontrado.

6.37 *Achar o valor mínimo em um array.* Escreva uma função recursiva `recursiveMinimum` que apanhe um array de inteiros e o tamanho do array como argumentos e retorne o menor elemento do array. A função deverá encerrar o processamento e retornar quando receber um array de um elemento.

Seção especial: Sudoku

O Sudoku passou a ser popular por volta de 2005. Hoje, quase todos os principais jornais publicam um quebra-cabeça de Sudoku diariamente. Players de jogos portáteis permitem que você jogue a qualquer hora e em qualquer lugar, criando desafios por demanda em vários níveis de dificuldade. Não deixe de visitar nosso Sudoku Resource Center em www.deitel.com/sudoku (em inglês) para obter downloads, tutoriais, livros, e-books e outros que o ajudarão a dominar o jogo. E, para os mais corajosos, experimentem resolver Sudokus cruelmente difíceis, com jogadas

intrincadas, um Sudoku circular e uma variante do quebra-cabeça com cinco grades interligadas. Assine nosso boletim gratuito, o *Deitel® Buzz Online*, para obter notas sobre atualizações em nosso Sudoku Resource Center e outros Deitel Resource Centers em www.deitel.com, que oferecem jogos, quebra-cabeças e outros projetos de programação interessantes.

Um quebra-cabeça de Sudoku completo é uma grade de 9×9 (ou seja, um array bidimensional) em que os dígitos de 1 a 9 aparecem uma, e somente uma, vez em cada linha, em cada coluna e

em cada uma das nove grades de 3×3 . Na grade de 9×9 parcialmente preenchida na Figura 6.27, a linha 1, a coluna 1 e a grade de 3×3 no canto superior esquerdo do quadro contêm os dígitos de 1 a 9 apenas uma vez. Usamos as convenções de numeração de linha e coluna do array bidimensional em C, mas ignoramos a linha 0 e a coluna 0, em conformidade com as convenções da comunidade Sudoku.

O jogo de Sudoku típico oferece muitas células preenchidas e muitos espaços, normalmente organizados em um padrão simétrico, como é comum nas palavras cruzadas. A tarefa do jogador é preencher os espaços para completar o quebra-cabeça. Alguns quebra-cabeças são fáceis de resolver; alguns são muito difíceis, exigindo estratégias de solução sofisticadas.

No Apêndice D, Programação de jogos: solução do Sudoku, discutimos diversas estratégias de solução simples e sugerimos o que fazer quando elas falharem. Também apresentamos diversas técnicas para programar criadores e solucionadores de quebra-cabeça Sudoku em C. Infelizmente, a C padrão não inclui gráficos ou capacidades de interface gráfica do usuário (GUI), de modo que nossa apresentação do quadro não será tão elegante quanto poderia ser em Java e em outras linguagens de programação que aceitam essas capacidades. Você pode querer refazer seus programas de Sudoku depois de ler o Apêndice E, Programação de jogos usando a biblioteca de C Allegro. Allegro, que não faz parte da C padrão, oferece capacidades que o ajudarão a acrescentar gráficos, e até mesmo sons, a seus programas de Sudoku.

	1	2	3	4	5	6	7	8	9
1	5	1	3	4	9	7	6	2	8
2	4	6	8						
3	7	9	2						
4	2								
5	9								
6	3								
7	8								
8	1								
9	6								

Figura 6.27 ■ Grade 9×9 de Sudoku parcialmente preenchida. Observe as nove grades 3×3 .